

Módulo 3: Evolução Progressiva de SPA no Canvas de IA

Disciplina: Programação Orientada a Objetos (POO)

Curso: Superior de Tecnologia em Sistemas para Internet

Estudante: Robert Vinicius Silva Macedo

Apresentação da Atividade

No Módulo 2, você aprendeu o **Framework C.T.R.E.** para criar prompts estruturados e profissionais. Agora, no Módulo 3, você aplicará esse mesmo framework para dominar um dos fluxos de trabalho mais produtivos do mercado: a **evolução incremental de software** utilizando o ambiente interativo do **Canvas**.

Em vez de criar uma aplicação inteira do zero, você guiará a IA de forma modular e progressiva, adicionando novas camadas de funcionalidade a um sistema já existente, sem perder o contexto.

Construiremos o **Terminal de Vendas Interativo (PDV)** e **Dashboard do ElectroVibe** em 4 etapas sucessivas de evolução do código.

Regras do Jogo no Canvas:

Comece uma **nova conversa** na IA.

Ative o modo **Canvas/Preview** para visualizar as alterações de interface em tempo real à medida que avança pelas etapas.

Mantenha-se no mesmo chat do início ao fim! Para que a evolução incremental funcione, a IA precisa do histórico do código anterior. Não mude de conversa entre as etapas.

ETAPA 1: O Núcleo do PDV (Interface e Catálogo)

Nesta primeira etapa, vamos construir a base visual e estrutural do nosso Ponto de Venda (PDV), exibindo os produtos em um grid interativo e preparando a estrutura de POO em memória.

Prompt de Envio - Etapa 1 (Framework C.T.R.E.):

"[CONTEXTO]

Aja como um Programador Frontend Sênior e Arquiteto de Software. Estou iniciando o desenvolvimento de uma aplicação web interativa em um arquivo único.

[TAREFA]

Crie o protótipo inicial de uma única página (SPA) em um único arquivo HTML para o Terminal de Vendas (PDV) da loja 'ElectroVibe'.

[REGRAS / RESTRIÇÕES]

Use Tailwind CSS (via CDN) para a estilização completa.

Garanta um tema dark moderno de alta qualidade estética (fundo principal em #0f172a, cards de produtos em #1e293b e texto claro #f8fafc).

O layout deve conter um cabeçalho simples com o nome 'ElectroVibe - Terminal de Vendas'.

Crie um Grid responsivo de 3 colunas para o catálogo de produtos. Cada card de produto deve ter uma imagem ilustrativa (use URLs do Unsplash), nome, preço e um botão elegante rotulado 'Adicionar'.

A barra lateral (aside) deve representar o cupom fiscal do carrinho atual. Neste momento, ela será estática, simulando apenas uma lista de itens adicionados de forma visual.

[ESPECIFICAÇÕES DE CÓDIGO (POO)]

O comportamento do sistema deve ser orientado a objetos em JavaScript Vanilla (ES6+).

Implemente a classe Produto com os seguintes atributos: id, nome, preco, categoria, estoque e imagem.

Implemente a classe TerminalVendas que possui um array de produtos no catálogo e gerencia a renderização dinâmica desse catálogo no HTML da página.

Instancie a classe TerminalVendas com 3 produtos eletrônicos simulados e renderize o catálogo automaticamente na inicialização da página."

Desafio de Análise - Etapa 1:

1. Questão de UI/UX: Como a IA lidou com o grid de produtos em termos de responsividade? O layout se adapta corretamente caso você diminua a janela do Canvas para simular uma tela móvel?

Sua resposta: A responsividade está boa. O layout se adapta corretamente na simulação de tela móvel.

2. Questão de POO: O botão "Adicionar" gerado pela IA já possui comportamento dinâmico? Como o evento de clique foi associado à lógica do JavaScript?

Sua resposta: Sim, o botão possui comportamento dinâmico. O evento de clicar foi associado ao eventListener('click').

ETAPA 2: O Carrinho Dinâmico e Controle de Estoque (Evolução 1)

Agora, na mesma conversa, vamos instruir a IA a evoluir o código adicionando a reatividade ao estoque e gerenciando o estado do carrinho por meio de uma classe dedicada.

Prompt de Envio - Etapa 2 (Framework C.T.R.E.):

"[CONTEXTO]

Continue agindo como o Arquiteto de Software Sênior responsável por este projeto. Vamos atualizar o código existente no Canvas.

[TAREFA]

Evolua o código do nosso arquivo HTML para adicionar um carrinho de compras dinâmico e o controle automático de estoque de produtos, alterando o estado do sistema de forma reativa.

[REGRAS / RESTRIÇÕES]

Atualize a barra lateral do carrinho de compras para listar os itens adicionados de forma dinâmica. O usuário deve ser capaz de alterar as quantidades dos itens ou remover um produto diretamente do carrinho.

Exiba o valor total a ser pago de forma altamente destacada, utilizando uma tipografia maior e uma cor de ênfase (como verde neon ou azul brilhante).

Adicione uma notificação visual flutuante (toast ou alert de interface temporário) que apareça no canto da tela sempre que um produto for adicionado ou removido.

[ESPECIFICAÇÕES DE CÓDIGO (POO)]

Implemente a classe Carrinho que gerencia um array privado de itens (cada item contendo o objeto do Produto e a respectiva quantidade adicionada).

A classe Carrinho deve expor os métodos necessários para gerenciar o estado da compra: adicionarItem(produto), removerItem(idProduto), calcularTotal() e esvaziar().

Ao adicionar um produto ao carrinho, o atributo estoque do respectivo objeto Produto deve ser decrementado em 1. Se o estoque desse produto chegar a 0, o botão 'Adicionar' do card do produto deve ficar visualmente desabilitado e exibir o texto 'Esgotado'.

Se um produto for removido do carrinho, seu estoque no catálogo deve ser restabelecido proporcionalmente."

Desafio de Análise - Etapa 2:

1. Questão de UI/UX (Microinterações): Como as notificações em formato "toast" e a mudança de estado do botão para "Esgotado" melhoram a experiência do usuário (UX) em comparação com as alterações estáticas da Etapa 1?

Sua resposta: Antes não tinha como o usuário saber visualmente se tinha algum item esgotado ou se o estoque de um item que comprava estava baixo. Agora a experiência do usuário melhorou bastante visualmente.

2. Questão de POO (Mudança de Estado): Como as classes Carrinho e TerminalVendas compartilham a referência dos objetos de Produto para manter o estoque consistente entre o catálogo e o carrinho?

Sua resposta: As classes compartilham os produtos por passagem por referência. O carrinho não cria uma cópia do produto; ele recebe o endereço de memória do objeto original que está no catálogo. Como ambos os componentes apontam para o mesmo local, qualquer alteração na propriedade ".estoque" feita por um deles é vista instantaneamente pelo outro.

ETAPA 3: O Dashboard Analítico com Gráficos SVG (Evolução 2)

Daremos mais inteligência ao sistema. Criaremos um painel de estatísticas que exibe as métricas de venda e renderiza gráficos usando SVGs desenhados dinamicamente pelas classes de POO.

Prompt de Envio - Etapa 3 (Framework C.T.R.E.):

"[CONTEXTO]

Continue agindo como o Arquiteto de Software Sênior do projeto. Estamos implementando um módulo analítico no sistema atual.

[TAREFA]

Adicione um Painel de Métricas (Dashboard) abaixo do terminal de vendas, exibindo o desempenho comercial do PDV em tempo real com gráficos dinâmicos.

[REGRAS / RESTRIÇÕES]

O painel deve ser inserido abaixo da seção principal do PDV de forma limpa e organizada.

Crie 3 'Cards de Indicadores' visuais:

Total de Vendas Confirmadas (em R\$).

Total de Itens Vendidos (unidades).

Alerta de Estoque Baixo (listando o nome dos produtos com quantidade em estoque menor que 2).

Adicione um gráfico de barras vertical simples representando as vendas por categoria de produto (ex: Smartphones, Áudio, Acessórios).

Atenção: As barras do gráfico devem ser elementos <svg> inline gerados dinamicamente, mantendo o estilo visual dark modern do sistema.

[ESPECIFICAÇÕES DE CÓDIGO (POO)]

Adicione um método finalizarVenda() à classe Carrinho. Ao clicar no botão 'Finalizar Venda' na barra lateral, limpe o carrinho e registre a venda em um histórico persistido em memória na classe TerminalVendas.

A classe TerminalVendas deve expor métodos de agregação para calcular os indicadores do Dashboard e reconstruir/gerar as tags SVG do gráfico com base no histórico de vendas confirmadas.

Garanta que o fluxo de atualizar a tela (métodos de renderização do catálogo e do painel) seja disparado reativamente sempre que uma venda for concluída."

Desafio de Análise - Etapa 3:

1. Questão de UI/UX (Visualização de Dados): O gráfico em SVG gerado na tela é de fácil leitura? De que forma o uso de cores ou rótulos auxilia um gerente de loja a tomar decisões rápidas?

Sua resposta:

O gráfico gerado é de fácil leitura, o gerente pode ver que tem um item com pouco estoque rapidamente por causa do ícone que está piscando em vermelho. E também pode ver qual produto está com maior faturamento.

2. Questão de POO (Responsabilidade Única): A classe TerminalVendas passou a acumular muitas funções (gerenciar catálogo, gerenciar vendas finalizadas e desenhar gráficos SVG). Como você reorganizaria essas classes no futuro para respeitar melhor o princípio de responsabilidade única?

Sua resposta: Eu iria criar classes diferentes com cada uma tendo sua responsabilidade única.

ETAPA 4: Regras de Negócio e Cupom de Impressão (Evolução Final)

Para fechar o nosso sistema de forma profissional, adicionaremos um sistema de descontos flexível às nossas regras de negócio orientadas a objetos e criaremos um leiaute de impressão para emissão de cupom fiscal.

Prompt de Envio - Etapa 4 (Framework C.T.R.E.):

"[CONTEXTO]

Atue como o Arquiteto de Software Sênior concluindo o ciclo de desenvolvimento do terminal de vendas.

[TAREFA]

Evolua o sistema do Canvas adicionando a funcionalidade de aplicação de descontos cumulativos ou cupons de desconto às regras do carrinho e crie um modo de emissão de cupom fiscal para impressão física.

[REGRAS / RESTRIÇÕES]

Adicione um campo de texto para cupom de desconto na barra lateral do carrinho e um botão 'Aplicar'.

Adicione um botão 'Imprimir Cupom' no carrinho de compras. Ao ser clicado, ele deve abrir um modal contendo a simulação do cupom fiscal com visualização formatada para impressão (fundo totalmente branco, texto em preto e bordas pontilhadas estilo cupom térmico).

Implemente uma diretiva de estilo CSS @media print específica para que, quando o usuário mandar imprimir o documento usando os atalhos do navegador (Ctrl+P / Cmd+P), apenas o modal do cupom apareça no papel, ocultando de forma transparente todo o restante da interface dark do terminal e do dashboard.

[ESPECIFICAÇÕES DE CÓDIGO (POO)]

Na classe Carrinho, adicione propriedades para gerenciar o valor ou percentual de desconto ativo (ex: descontoAtivo).

Crie os métodos aplicarCupom(codigoCupom) (para validar cupons em memória, como 'ELETRO10' para 10% de desconto) e calcularTotalComDesconto(). O valor total exibido no dashboard e no cupom deve refletir essa regra de negócio.

Implemente o método gerarCupomHTML() na classe Carrinho que retorne a string de marcação HTML estilizada com as informações consolidadas da venda (itens comprados, valor bruto, desconto aplicado e valor líquido final) para preenchimento dinâmico do modal."

Desafio de Análise - Etapa 4:

1. Questão de UI/UX (Design de Impressão): Por que as regras de estilização voltadas para a impressão física (papel) exigem uma abordagem visual drasticamente diferente da estilização de interfaces focadas em telas de computadores (como o nosso tema escuro)?

Sua resposta: Por que imprimir o papel em branco é mais eficiente em questão de gasto de tinta e reflexão da luz para legibilidade.

2. Questão de Programação Orientada a Objetos (Responsabilidade e Coesão): Analise a implementação do método gerarCupomHTML() na classe Carrinho. Sob a perspectiva de POO, por que a responsabilidade de calcular os descontos e formatar o cupom fiscal pertence à classe Carrinho e não à classe Produto ou ao controle geral da interface? Como isso se relaciona com o princípio de alta coesão e especialista na informação?

Sua resposta: O Carrinho é o único que conhece a lista de itens, quantidades e o cupom ativo. O Produto é isolado e não tem essa visão global da venda. A interface apenas pede o cupom pronto e o exibe. Se a regra do desconto mudar, altera-se apenas o Carrinho, sem quebrar a tela ou o dashboard. E também mantém o cálculo de descontos e a estrutura do cupom juntos no domínio da venda. Isso impede que regras de negócio comerciais e fiscais fiquem espalhadas ou "vazem" para a interface de usuário.